

PLUG: Power-Law Upgraded GRU

Architecture, implementation, and preliminary experiments

Contact: rickeyernest1990@gmail.com

Publication date: 31 August 2026

Contents

Overview	2
1. PLUG at a glance	2
2. Inverse-lag history	3
3. Why normalise the history?	4
4. The augmented GRU input	5
5. Standard GRU equations used by PLUG	5
6. Reset gate	6
7. Update gate	6
8. Candidate state	6
9. Final state update	6
10. Complete PLUG specification	7
11. Parameter dimensions and parameter matching	7
12. Why the power-law path may help long-range learning	8
13. Efficient history calculation with FFT	8
Critical alignment rule	9
Streaming note	10
14. Minimal PyTorch-style reference implementation	10
15. Numerical implementation checks	11
16. Preliminary experiments: PLUG versus GRU	11
16.1 Long recall under continuously changing distractors	11
16.2 One-bit retention range	12
16.3 Gradient-horizon diagnostic	13
16.4 Hostile-clutter stress test	14
16.5 Selective two-item memory	14

16.6 Corrected copy-memory screen	14
16.7 Relational fact-binding screen	15
16.8 Classic Adding Problem	15
16.9 Training-time overhead in the CPU screening runs	15
17. What the experiments currently support	16
18. Reproducibility checklist	16
19. Short implementation-ready definition	16
20. Publication, contact and licensing	17
Licensing	17

Licensing warning - noncommercial use. This publication uses a split licence. The **documentation, explanatory text, figures, tables, and other copy-rightable presentation in this PDF** are licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#). **Source code and code listings implementing PLUG** are licensed separately under the [PolyForm Noncommercial License 1.0.0](#). Please review the applicable licence before copying, adapting, distributing, or reusing material from this publication.

Overview

PLUG (Power-Law Upgraded GRU) is an experimental augmentation of a conventional GRU. It leaves the GRU’s internal recurrent equations unchanged and adds one additional input channel: a **normalised inverse-lag history** of previous input representations.

The aim is deliberately simple. A standard GRU normally receives

$$x_t, \quad h_{t-1}.$$

PLUG gives the same GRU an augmented input

$$e_t = [x_t; P_t],$$

where P_t is a deterministic power-law summary of previous inputs using the weights

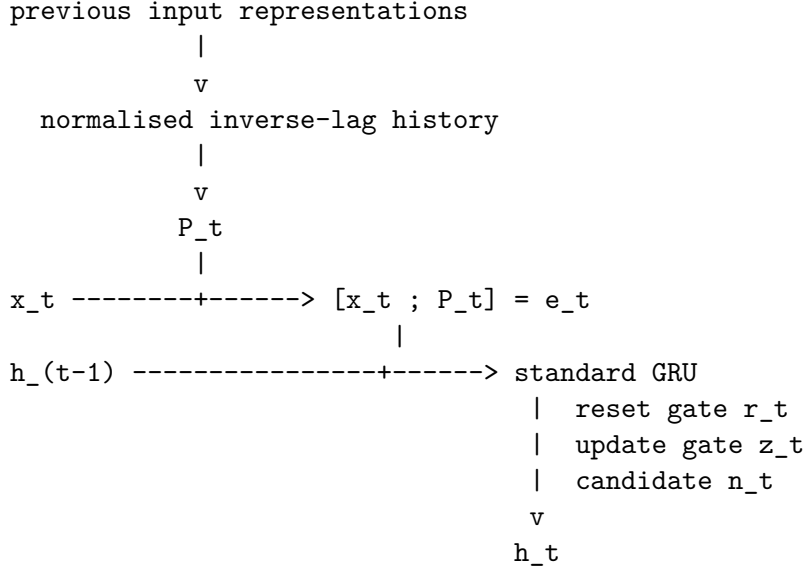
$$1, \quad \frac{1}{2}, \quad \frac{1}{3}, \quad \frac{1}{4}, \quad \frac{1}{5}, \dots$$

The recurrent core itself remains the standard GRU implementation.

Experimental status. PLUG is still experimental. The results later in this document are controlled screening experiments, not peer-reviewed evidence of general superiority over GRU. The strongest evidence so far concerns long persistence under changing interference.

1. PLUG at a glance

PLUG is easiest to understand as a wrapper around a standard GRU:



The only architectural change relative to the baseline GRU is

$$e_t = [x_t; P_t].$$

The tested implementation used `torch.nn.GRU` unchanged after this concatenation. In the original experiment logs, PLUG was named **GRU+H**.

2. Inverse-lag history

Let the current input representation be

$$x_t \in \mathbb{R}^{d_x}.$$

Define the harmonic number

$$\mathcal{H}_t = \sum_{k=1}^t \frac{1}{k}.$$

PLUG constructs

$$P_t = \frac{\sum_{k=1}^t \frac{x_{t-k}}{k}}{\sum_{k=1}^t \frac{1}{k}}$$

with

$$P_0 = 0.$$

Equivalently,

$$P_t = \frac{1}{\mathcal{H}_t} \sum_{k=1}^t \frac{x_{t-k}}{k}.$$

A critical implementation rule is that P_t contains **previous input representations only**. The current input x_t is not included.

The first few history vectors are

$$P_0 = 0,$$

$$P_1 = x_0,$$

$$P_2 = \frac{x_1 + \frac{1}{2}x_0}{1 + \frac{1}{2}},$$

and

$$P_3 = \frac{x_2 + \frac{1}{2}x_1 + \frac{1}{3}x_0}{1 + \frac{1}{2} + \frac{1}{3}}.$$

The underlying lag kernel is

$$\boxed{K(k) = \frac{1}{k}},$$

which is a power-law kernel with exponent 1.

3. Why normalise the history?

Without normalisation,

$$\sum_{k=1}^t \frac{x_{t-k}}{k}$$

can slowly increase in scale as the sequence grows. Dividing by $\mathcal{H}_t$ makes the coefficients sum to one:

$$\sum_{k=1}^t \frac{1/k}{\mathcal{H}_t} = 1.$$

Thus P_t behaves like a weighted historical average rather than an ever-growing accumulated sum.

The harmonic number grows approximately as

$$\mathcal{H}_t \approx \ln t + \gamma,$$

where γ is the Euler-Mascheroni constant. An input k positions in the past therefore has normalised contribution approximately

$$\frac{1}{k(\ln t + \gamma)}.$$

4. The augmented GRU input

PLUG forms

$$e_t = [x_t; P_t].$$

Because both x_t and P_t have dimension d_x ,

$$e_t \in \mathbb{R}^{2d_x}.$$

The GRU then receives

$$e_t, \quad h_{t-1}$$

instead of

$$x_t, \quad h_{t-1}.$$

Nothing else in the recurrent core needs to be redesigned.

5. Standard GRU equations used by PLUG

The experiments used the standard PyTorch GRU implementation. With e_t replacing the ordinary GRU input, one layer computes

$$r_t = \sigma(W_{ir}e_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

$$z_t = \sigma(W_{iz}e_t + b_{iz} + W_{hz}h_{t-1} + b_{hz})$$

$$n_t = \tanh(W_{in}e_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn}))$$

and

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

Here:

- r_t is the standard GRU reset gate;
- z_t is the standard GRU update gate;
- n_t is the standard GRU candidate state;
- h_t is the new recurrent state.

The important point is that PLUG does **not** introduce a third learned gate or replace the GRU update rule. The power-law history enters only through e_t .

6. Reset gate

The reset gate is

$$r_t = \sigma(W_{ir}e_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}).$$

Because $e_t = [x_t; P_t]$, the ordinary GRU reset gate can now respond to both the current input and the inverse-lag history.

7. Update gate

The update gate is

$$z_t = \sigma(W_{iz}e_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}).$$

As in a standard GRU, z_t determines the interpolation between the previous state and the candidate state:

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

PLUG therefore retains the familiar GRU memory mechanism while giving its gates access to a slowly decaying historical signal.

8. Candidate state

The tested PyTorch GRU uses the reset-after form

$$n_t = \tanh(W_{in}e_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})).$$

This detail matters for exact reproduction. A different GRU convention can be mathematically related but is not identical to the framework implementation used in the experiments.

9. Final state update

PLUG uses the unchanged standard GRU update

$$\boxed{h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.}$$

When z_t is close to one, the old state is strongly retained. When z_t is closer to zero, more of the new candidate is admitted.

The difference from a conventional GRU is not this update equation; it is the richer input e_t used to construct the gates and candidate.

10. Complete PLUG specification

For an independent implementation, PLUG is fully specified by

$$P_0 = 0,$$

$$P_t = \frac{\sum_{k=1}^t x_{t-k}/k}{\sum_{k=1}^t 1/k},$$

$$e_t = [x_t; P_t],$$

followed by an otherwise unchanged standard GRU:

$$r_t = \sigma(W_{ir}e_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}),$$

$$z_t = \sigma(W_{iz}e_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}),$$

$$n_t = \tanh(W_{in}e_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})),$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

For zero initialisation,

$$h_{-1} = 0.$$

The basic PLUG definition contains no attention, no external key-value memory, no learned decay exponent, and no custom recurrent gate beyond the GRU itself.

11. Parameter dimensions and parameter matching

Since

$$e_t \in \mathbb{R}^{2d_x},$$

the input-side GRU matrices have dimensions

$$W_{ir}, W_{iz}, W_{in} \in \mathbb{R}^{d_h \times 2d_x}.$$

The recurrent matrices have dimensions

$$W_{hr}, W_{hz}, W_{hn} \in \mathbb{R}^{d_h \times d_h}.$$

The tested PyTorch implementation has two bias vectors per gate family, so the one-layer recurrent parameter count is

$$\boxed{3d_h(2d_x + d_h) + 6d_h}.$$

A standard GRU with the same hidden width would have fewer parameters because its input width is d_x rather than $2d_x$. Therefore fair experiments should parameter-match by adjusting hidden width rather than simply using the same d_h .

That rule was followed in the screening experiments below.

12. Why the power-law path may help long-range learning

An ordinary GRU has to learn to preserve useful information through its recurrent dynamics. PLUG adds an additional route because an old input contributes directly to later P_t vectors.

For an input k steps old,

$$\boxed{\frac{\partial P_t}{\partial x_{t-k}} = \frac{1}{k\mathcal{H}_t}I}.$$

For the first input x_0 ,

$$\boxed{\frac{\partial P_t}{\partial x_0} = \frac{1}{t\mathcal{H}_t}I}.$$

Since $\mathcal{H}_t \sim \ln t$, this direct contribution decreases approximately as

$$\boxed{\frac{1}{t \ln t}}.$$

This does not guarantee that the task-relevant gradient of the entire network will stay large, and it does not imply unlimited memory. It means that the added history path itself has a polynomially decaying direct dependency rather than requiring an old input to survive only through a long recurrent chain.

13. Efficient history calculation with FFT

The numerator

$$N_t = \sum_{k=1}^t \frac{x_{t-k}}{k}$$

is a **linear causal convolution**. Define the fixed kernel

$$q = \left[0, 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots\right].$$

Then, for the required causal samples,

$$N = X * q.$$

When FFTs are used, the convolution must be **zero-padded** so that the FFT computes a linear convolution rather than a circular one. For an input sequence of length T , choose an FFT length

$$L \geq 2T - 1$$

(often the next convenient fast FFT length), pad both X and q with zeros to length L , and compute

$$\widehat{N} = \text{IFFT}_L(\text{FFT}_L(X_{\text{pad}}) \odot \text{FFT}_L(q_{\text{pad}})).$$

Keep the first T causal samples:

$$N_t = \widehat{N}_t, \quad 0 \leq t < T.$$

Finally set

$$P_0 = 0,$$

and for $t \geq 1$,

$$P_t = \frac{N_t}{\mathcal{H}_t}.$$

A direct all-timestep implementation is approximately $O(T^2)$, whereas the zero-padded FFT convolution is approximately

$$O(T \log T).$$

The FFT does not redefine PLUG. It is a faster numerical evaluation of the same inverse-lag history, apart from ordinary floating-point round-off.

Critical alignment rule

The kernel must start with zero:

```
q[0] = 0
q[1] = 1
q[2] = 1/2
q[3] = 1/3
...
```

That leading zero ensures that the current input x_t is excluded from P_t . The zero-padding rule above is equally important: without sufficient zero-padding, an FFT implementation performs circular convolution and contaminates the history with wrap-around terms.

Streaming note

For full-sequence training, the FFT form is natural because the complete sequence is already available. For token-by-token autoregressive generation, recomputing a full FFT after every token would be inefficient. An exact streaming implementation should use an online or partitioned convolution strategy rather than a new full-sequence FFT at every step.

14. Minimal PyTorch-style reference implementation

The following code shows the defining PLUG change as plainly as possible. The slow history function is useful as a correctness reference; an FFT version can replace it after verification.

```
import torch
import torch.nn as nn

def inverse_lag_history_reference(x):
    """
    x: [batch, time, input_size]

    P[:, t] contains the normalised inverse-lag
    history of inputs strictly before timestep t.
    """
    B, T, D = x.shape
    P = torch.zeros_like(x)

    for t in range(1, T):
        numerator = torch.zeros_like(x[:, 0])
        denominator = 0.0

        for k in range(1, t + 1):
            numerator = numerator + x[:, t-k] / k
            denominator += 1.0 / k

        P[:, t] = numerator / denominator

    return P

class PLUG(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()

        # The standard GRU is unchanged internally.
        # Its input width is doubled because it receives [x_t ; P_t].
        self.gru = nn.GRU(
            input_size=2 * input_size,
            hidden_size=hidden_size,
            batch_first=True
        )

    def forward(self, x, h0=None):
```

```

P = inverse_lag_history_reference(x)
e = torch.cat([x, P], dim=-1)
y, h = self.gru(e, h0)
return y, h

```

With a verified FFT history implementation, the defining forward pass becomes conceptually

```

P = inverse_lag_history_fft(x)
e = torch.cat([x, P], dim=-1)
y, h = self.gru(e, h0)

```

This is why PLUG is intended to be a low-friction GRU augmentation: the tokenizer, embedding stack, training loop, GRU backend, output head, optimiser and generation logic can all remain conventional.

An already-trained GRU checkpoint is not literally shape-compatible without adapting its input weights, because the GRU input width changes from d_x to $2d_x$. “Plug” refers to the architecture and software pathway, not to zero-conversion checkpoint compatibility.

15. Numerical implementation checks

The inverse-lag history used by PLUG and ILRM was independently checked against direct summation.

- Direct summation versus FFT history in float64: maximum absolute error no larger than approximately 8.88×10^{-16} in the focused checks.
- During the final document recheck, the explicitly zero-padded FFT linear convolution was again compared with the direct history definition on random float64 sequences; the maximum absolute difference was 6.66×10^{-16} .
- The reset-after GRU equations printed in this document were independently evaluated with the exact weights and biases from a float64 `torch.nn.GRUCell`; the manual equations and framework output agreed to a maximum absolute difference of 3.33×10^{-16} in that check.
- At the framework level, PLUG itself required no custom recurrent arithmetic: the tested model passed the concatenated input $[x_t; P_t]$ directly into `torch.nn.GRU`.

These checks verify both pieces of the implementation: the inverse-lag input is calculated as intended, and the displayed PLUG recurrence matches the framework GRU convention used in the experiments.

16. Preliminary experiments: PLUG versus GRU

The model now called **PLUG** was named **GRU+H** in the original experiment logs. Only the name has changed; the implementation and reported values are unchanged.

The tables below intentionally show **PLUG and the standard GRU only**.

16.1 Long recall under continuously changing distractors

One of eight symbols was presented, followed by changing random distractor tokens, then a query asking for the original symbol. Chance accuracy was 12.5%.

The models were trained only on delays from **16 to 64 steps** and tested much farther without re-training. The focused confirmation used **five paired training seeds**, identical fixed evaluation examples, common optimiser settings, and parameter-matched models:

- GRU: 3,868 trainable parameters

- PLUG: 3,908 trainable parameters (1.03% more)

Distance	GRU	PLUG
64	12.8 +/- 1.2%	100.0 +/- 0.0%
128	12.7 +/- 1.5%	100.0 +/- 0.0%
256	12.0 +/- 2.0%	99.4 +/- 1.3%
512	12.5 +/- 1.8%	97.8 +/- 4.9%
1,024	12.1 +/- 1.5%	95.8 +/- 5.8%
2,048	11.9 +/- 1.3%	93.5 +/- 9.1%
4,096	12.5 +/- 1.3%	86.6 +/- 9.2%

At distance 1,024, the paired PLUG-minus-GRU difference was about **+83.7 percentage points**, with a preliminary paired 95% interval of approximately **+76.0 to +91.4 points**.

Learning trajectory at an unseen distance of 1,024

Updates	GRU	PLUG
300	12.0%	13.5%
600	11.6%	13.5%
900	14.1%	30.3%
1,200	13.6%	66.3%
1,800	12.1%	95.5%

The standard GRU remained around chance, while PLUG eventually learned a solution that extrapolated far beyond the training lengths.

Exploratory ultra-long 8-way recall

The same trained models were also probed beyond the well-sampled 4,096-step endpoint:

Distance	GRU	PLUG
8,192	9.7%	84.7%
16,384	16.9%	76.2%

These two points are exploratory: only 64 unique sequences were used at 8,192 and 32 at 16,384.

16.2 One-bit retention range

A separate test stored one binary bit at the start of a sequence, inserted changing distractors, and queried the original bit at the end. Chance was 50%.

Training delays remained 16-64. The smaller parameter-matched models used:

- GRU: 1,252 parameters
- PLUG: 1,272 parameters (1.60% more)

Three independently trained paired seeds were tested.

Distance	GRU	PLUG	Unique fixed examples
64	50.2%	100%	512
256	50.5%	100%	512
1,024	50.0%	100%	512
4,096	50.3%	100%	512
8,192	51.0%	100%	64
16,384	51.0%	100%	32
32,768	43.8%	100%	16
65,536	54.2%	100%	8

The strongest well-sampled statement is **100% PLUG accuracy at 4,096 steps on 512 fixed examples across all three trained models after training only up to delay 64**. The longer distances are exploratory lower bounds because the number of unique sequences was deliberately reduced to control CPU cost.

16.3 Gradient-horizon diagnostic

A separate diagnostic used the already-trained 1,800-update checkpoints without further tuning. GRU and PLUG received identical sequences, and gradients were evaluated in float64. The diagnostic scalar was the raw correct-class logit margin.

- **Token gradient:** $\|\partial m_T / \partial x_0\|_2$
- **Initial-state gradient:** $\|\partial m_T / \partial h_0\|_2$

Five-seed means after training were:

Distance	GRU token grad	PLUG token grad	GRU h_0 grad	PLUG h_0 grad
16	1.41e-05	7.31e-01	6.07e-05	1.68e+00
32	1.60e-08	5.68e-01	6.74e-08	1.15e+00
64	2.03e-14	5.97e-01	6.61e-14	1.00e+00
128	2.52e-26	5.39e-01	8.01e-26	1.04e+00
256	2.03e-49	7.41e-01	8.94e-49	1.98e+00
512	2.63e-95	4.57e-01	1.16e-94	1.17e+00
1,024	numerical underflow	6.55e-01	numerical underflow	1.27e+00
4,096	numerical underflow	4.98e-01	numerical underflow	1.00e+00

At random initialisation, PLUG still had a strongly vanishing recurrent-state gradient, so the power-law input does not magically make the initial GRU Jacobian stable. However, PLUG already possessed the direct token-to-history pathway. At 1,024 steps, its mean initial token gradient was approximately 9.18×10^{-5} versus approximately 2.80×10^{-145} for the GRU.

After training, PLUG had learned recurrent dynamics with order-one-scale task-relevant h_0 gradients through at least 4,096 steps in this diagnostic. The later accuracy decline is therefore not explained solely by a classical vanished gradient at 4,096; interference, signal dilution and finite capacity are plausible limiting factors.

16.4 Hostile-clutter stress test

Fresh examples at distance 1,024 deliberately made the distractors misleading. These were five-seed evaluations of previously trained parameter-matched models. Chance was 12.5%.

Distractor pattern	GRU	PLUG
Random changing distractors	12.7%	95.4%
Wrong symbol immediately before query	8.0%	95.0%
Final 25% all the same wrong symbol	5.5%	87.7%
Every distractor the same wrong symbol	4.9%	73.2%

This argues against a simple strategy such as following the most recent or most frequent symbol.

16.5 Selective two-item memory

A more demanding experiment tested whether the model could preserve an early relevant value while later accepting a second relevant value.

Each sequence contained:

A_MARK, A_VALUE, clutter..., B_MARK, B_VALUE, clutter..., QUERY_A or QUERY_B

Values and clutter used eight symbols, so chance was 12.5%. The five-seed parameter-matched models used 3,904 GRU parameters and 3,944 PLUG parameters (1.02% more).

Query A: preserve the early item after B arrives

Distance	GRU	PLUG
64	12.9%	99.3%
256	13.1%	94.1%
1,024	12.4%	75.0%

Query B: accept the later relevant item

Distance	GRU	PLUG
64	13.0%	64.2%
256	12.5%	54.5%
1,024	12.7%	37.7%

The second-slot result had substantial seed-to-seed variability. This is evidence that the history-augmented GRU can preserve and later accept multiple task-relevant events better than the matched GRU in this screen, but it is not evidence of robust arbitrary multi-item memory.

16.6 Corrected copy-memory screen

A corrected copy-memory experiment trained the loss on the actual recall segment so that blank-token prediction could not dominate the objective. At distance 1,000, after 1,800 updates, the three-seed sealed-test token recall was:

- GRU: **15.9** +/- **4.1%**
- PLUG: **25.2** +/- **0.6%**

The models were closely parameter matched (GRU 3,909; PLUG 3,945, or 0.92% more parameters for PLUG).

No model achieved exact 10-token sequence recall in this small-compute screen, so this is evidence of improved **partial token recall**, not a solved copy-memory benchmark.

16.7 Relational fact-binding screen

A small synthetic fact-binding benchmark used fresh person-colour-object assignments and then queried the correct relation. After longer training, the early apparent history advantage largely disappeared.

On the three-seed sealed 4-fact endpoint:

- GRU: **25.3** +/- **0.9%**
- PLUG: **24.8** +/- **1.6%**

At larger loads, the results were also very close:

- 8 facts: GRU 12.1%, PLUG 12.4%
- 12 facts: GRU 8.3%, PLUG 8.4%
- 16 facts: GRU 5.9%, PLUG 5.5%

The parameter counts were GRU 5,677 and PLUG 5,720 (0.76% more for PLUG).

This benchmark therefore **does not currently support a PLUG advantage**.

16.8 Classic Adding Problem

The Adding Problem provides another useful counterexample. After 1,800 updates, both models solved the task, but the GRU had the lower sealed-test MSE:

- GRU: **0.000362** +/- **0.000111**
- PLUG: **0.000607** +/- **0.000112**

Lower is better. The models were closely matched in parameter count (GRU 3,655; PLUG 3,697, or 1.15% more for PLUG).

This result does **not** support a general claim that adding the power-law history improves every GRU task.

16.9 Training-time overhead in the CPU screening runs

Because PLUG keeps the standard framework GRU and computes the history with FFT, its measured training overhead was modest in these small CPU experiments. This is implementation-dependent and should not be treated as a hardware-independent property.

Task	GRU train time	PLUG train time	Relative difference
Adding	25.64 s	25.79 s	+0.6%
Copy	32.18 s	31.74 s	-1.4%
Distractor	12.60 s	13.93 s	+10.6%
Fact binding	7.84 s	8.82 s	+12.5%
Focused 5-seed distractor run	26.26 s	28.75 s	+9.5%

The small negative copy-time difference is ordinary run-to-run noise, not evidence that PLUG is intrinsically faster than GRU. The useful observation is that the added history did not create an order-of-magnitude training slowdown in this implementation.

17. What the experiments currently support

Taken together, the screening experiments support a narrower and more defensible picture than “PLUG fixes GRU memory”.

Evidence currently favouring PLUG:

- much stronger retention under continuously changing distractors;
- large train-length extrapolation beyond the 16-64 training window;
- perfect performance on the well-sampled 4,096-step one-bit test;
- a direct inverse-lag token-gradient pathway that does not decay exponentially by construction;
- learned long-range recurrent dynamics in the trained gradient diagnostic;
- resistance to recent and repeated wrong-symbol clutter;
- improved selective two-item retention in the synthetic screen;
- better partial token recall on the corrected copy-memory screen;
- modest measured overhead while retaining the standard optimised GRU backend.

Results that do not favour PLUG:

- the classic Adding Problem favoured GRU at the final endpoint;
- the longer-trained relational fact-binding screen was essentially a tie and did not support a PLUG advantage;
- second-slot selective memory still had substantial seed-to-seed variability;
- the 8k-65k extreme-distance probes used progressively smaller test sets and are exploratory.

The current evidence suggests that PLUG’s most promising property is **long persistence under changing interference**, while preserving the engineering simplicity of a standard GRU.

18. Reproducibility checklist

An independent implementation should preserve the following points:

1. P_t is computed from **previous input representations**, never previous hidden states.
2. The history kernel is exactly $1/k$ and is normalised by the corresponding harmonic number.
3. $P_0 = 0$.
4. The augmented GRU input is exactly $e_t = [x_t; P_t]$.
5. The tested recurrent core is an otherwise unchanged standard PyTorch `nn.GRU`.
6. The tested GRU uses the framework reset-after candidate form shown in this document.
7. There is no third gate, no attention mechanism, no external memory and no learned power-law exponent in the basic PLUG definition.
8. If FFT is used, the convolution kernel begins with zero so that the current input is excluded from P_t , and both sequences are zero-padded to a length of at least $2T - 1$ so the FFT computes linear rather than circular convolution.
9. Parameter matching against GRU should adjust hidden width so total trainable parameters remain closely matched.
10. A new FFT implementation should first be checked against the direct float64 history definition before benchmarking.

19. Short implementation-ready definition

PLUG (Power-Law Upgraded GRU) augments a conventional GRU with a normalised inverse-lag history of previous input representations. At timestep t ,

$$P_t = \frac{\sum_{k=1}^t x_{t-k}/k}{\sum_{k=1}^t 1/k}, \quad P_0 = 0.$$

Form

$$\boxed{e_t = [x_t; P_t]}.$$

Feed e_t into an otherwise unchanged standard GRU. In the tested PyTorch form,

$$r_t = \sigma(W_{ir}e_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}),$$

$$z_t = \sigma(W_{iz}e_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}),$$

$$n_t = \tanh(W_{in}e_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})),$$

and

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}.$$

The single most important rule is:

PLUG changes the GRU input, not the GRU recurrence: compute the inverse-lag history from previous input representations, concatenate it with the current input, and feed the result into an otherwise unchanged standard GRU.

20. Publication, contact and licensing

Publication date: 31 August 2026

Contact: rickeyernest1990@gmail.com

Licensing

Licensing warning - noncommercial use. Different parts of this publication are licensed under different noncommercial licences. Use the licence that applies to the material you are reusing.

Documentation, explanatory text, figures and tables

The copyrightable documentation and presentation in this PDF are licensed under the [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#). This permits sharing and adaptation subject to the licence terms, including attribution and the noncommercial-use restriction.

Source code and code listings

Source code implementing **PLUG**, including implementation code and source-code listings reproduced in this document, is licensed separately under the [PolyForm Noncommercial License 1.0.0](#).

Commercial use

Neither of the licences above grants unrestricted commercial-use rights. For commercial licensing enquiries, contact rickeyernest1990@gmail.com.

Scope note

These licence notices apply to the copyrightable expression, documentation, figures, presentation, and software supplied with this work to the extent protected by applicable law. They are not intended to restrict mathematical facts, formulas, abstract ideas, or other material that is not subject to copyright protection. The licence notices do not themselves establish or imply any claim about patentability, novelty, or ownership of ideas that may be governed by separate law or prior art.